

fork() / exec() programs

Every program from the chat, with a block diagram of the parent/child relationship each one creates.

1. Basic fork — different text in parent and child

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

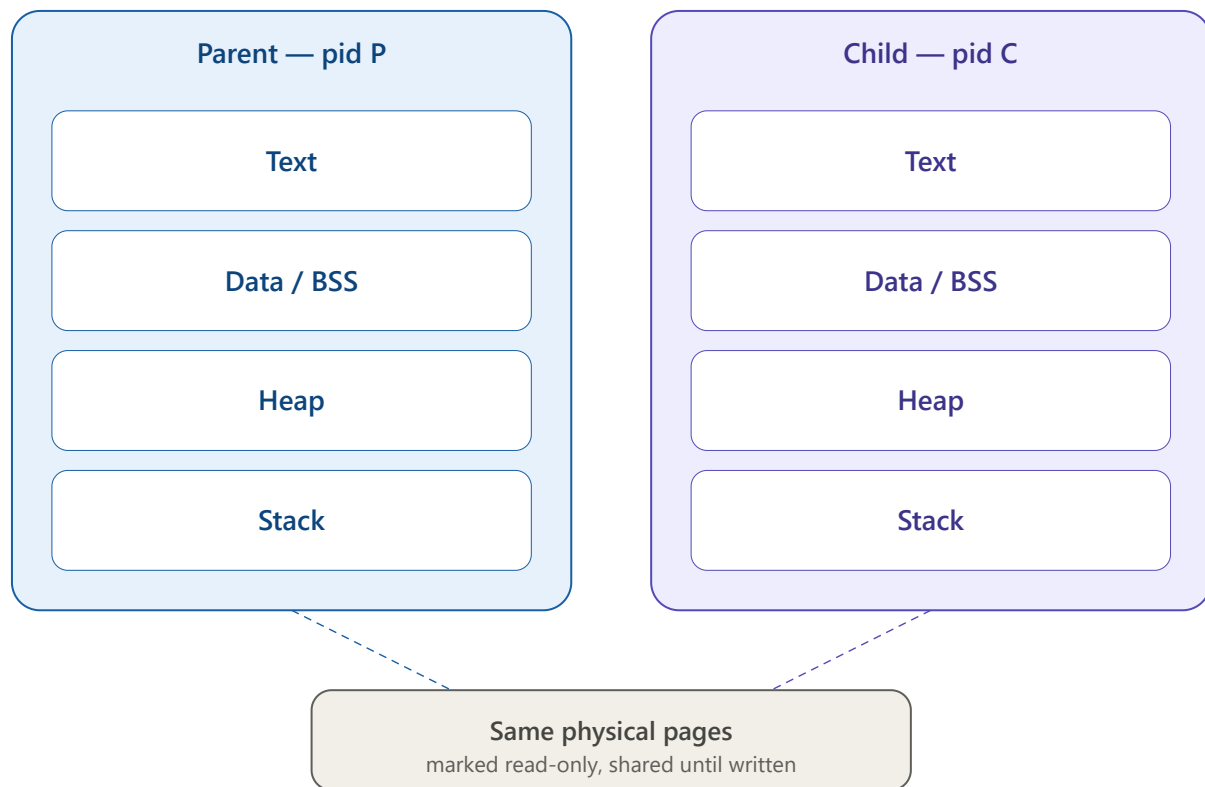
    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        wait(NULL);
        printf("Parent: child has exited\n");
    }

    return 0;
}
```

```
$ gcc -o forktest forktest.c
$ ./forktest
```

Virtual address space of parent and child, right after fork()

Two separate mm_structs, initially pointing at identical content



Both address spaces have the same content at the instant of `fork()` — same code, same globals, same heap, same stack. The kernel duplicates the *page tables* (so each has an independent mapping), but marks the underlying physical pages read-only and shared. The first time either process writes to a page (e.g. the child's `printf` buffer, or a local variable), a page fault triggers the kernel to copy just that one page — this is copy-on-write, and it's why `fork()` is cheap even for a process with a large heap.

2. Parent waits for a specific child pid

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
    }
}
```

```
    } else {
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);

        int status;
        pid_t exited_pid = waitpid(pid, &status, 0);

        if (exited_pid == pid) {
            if (WIFEXITED(status))
                printf("Parent: child %d exited with status %d\n", exited_pid, WEXITSTATUS(st
            else
                printf("Parent: child %d did not exit normally\n", exited_pid);
        }
    }

    return 0;
}
```

```
$ gcc -o forktest forktest.c
$ ./forktest
```

3. Child sleeps for 20 seconds

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: my pid = %d, parent pid = %d\n", getpid(), getppid());
        printf("Child: sleeping for 20 seconds...\n");
        fflush(stdout);
        sleep(20);
        printf("Child: woke up, exiting now\n");
    } else {
        printf("Parent: my pid = %d, child pid = %d\n", getpid(), pid);
        printf("Parent: waiting for child to finish...\n");
        fflush(stdout);

        int status;
        pid_t exited_pid = waitpid(pid, &status, 0);

        if (exited_pid == pid) {
```

```

        if (WIFEXITED(status))
            printf("Parent: child %d exited with status %d\n", exited_pid, WEXITSTATUS(st
        else
            printf("Parent: child %d did not exit normally\n", exited_pid);
    }
}

return 0;
}

```

```

$ gcc -o forktest forktest.c
$ ./forktest

```



Parent is off the runqueue (state S) for the full 20s — not spinning, just descheduled

4. Parent forks 3 children (10s, 20s, 30s), waits for all

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int sleep_times[3] = {10, 20, 30};
    pid_t child_pids[3];

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
            return 1;
        } else if (pid == 0) {
            printf("Child %d: pid = %d, sleeping for %d seconds\n",
                i, getpid(), sleep_times[i]);
            fflush(stdout);
            sleep(sleep_times[i]);
        }
    }
}

```

```

        printf("Child %d: pid = %d, woke up after %d seconds\n",
               i, getpid(), sleep_times[i]);
        return 0;
    } else {
        child_pids[i] = pid;
    }
}

printf("Parent: pid = %d, waiting for all children...\n", getpid());
fflush(stdout);

for (int i = 0; i < 3; i++) {
    int status;
    pid_t exited_pid = waitpid(child_pids[i], &status, 0);
    if (exited_pid == child_pids[i] && WIFEXITED(status)) {
        printf("Parent: child %d (pid %d) exited with status %d\n",
               i, exited_pid, WEXITSTATUS(status));
    }
}

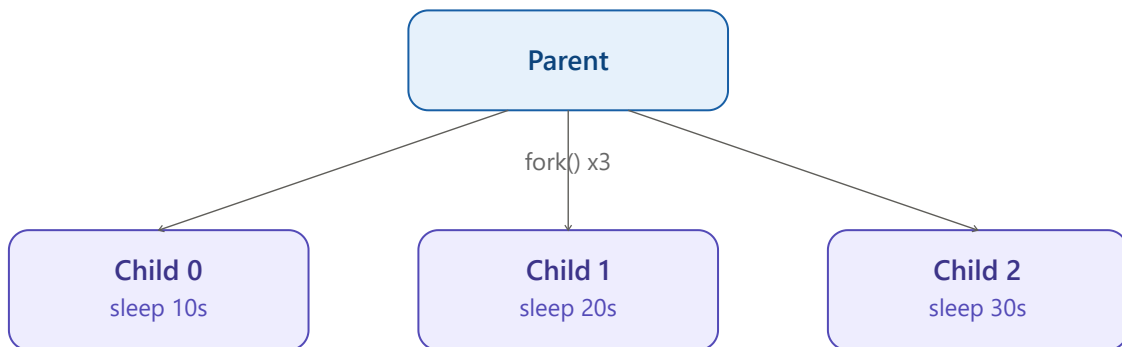
printf("Parent: all children have exited\n");
return 0;
}

```

```

$ gcc -o forktest forktest.c
$ ./forktest

```



Parent calls `waitpid(child_pids[i])` in order 0,1,2 — so print order is fixed even though child 0 finishes first anyway

5. rec_fork — parent → child1 → child2 → child3, each waits for its own child

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

```

```
int main() {
    printf("Original parent: pid = %d\n", getpid());
    fflush(stdout);

    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork failed");
        return 1;
    } else if (pid1 == 0) {
        // ---- child1 ----
        printf("Child1: pid = %d, parent pid = %d\n", getpid(), getppid());
        fflush(stdout);

        pid_t pid2 = fork();

        if (pid2 < 0) {
            perror("fork failed");
            return 1;
        } else if (pid2 == 0) {
            // ---- child2 ----
            printf("Child2: pid = %d, parent pid = %d\n", getpid(), getppid());
            fflush(stdout);

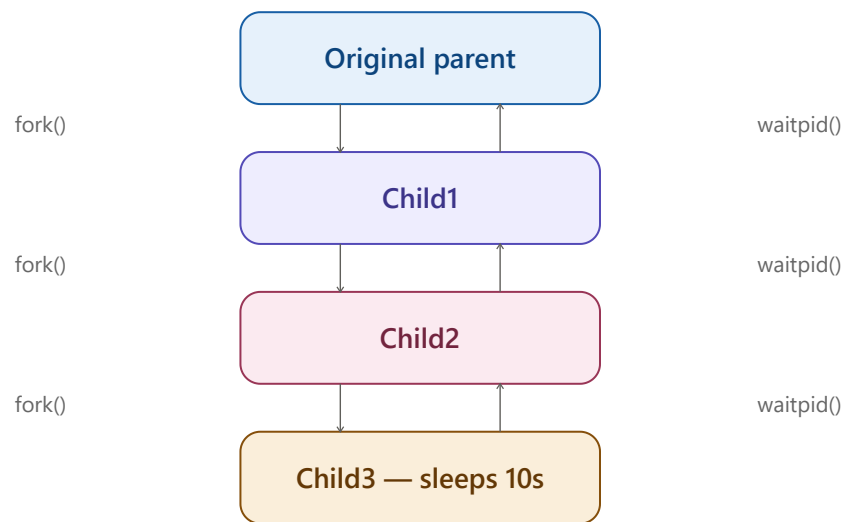
            pid_t pid3 = fork();

            if (pid3 < 0) {
                perror("fork failed");
                return 1;
            } else if (pid3 == 0) {
                // ---- child3 ----
                printf("Child3: pid = %d, parent pid = %d, sleeping 10s\n", getpid(), getppid());
                fflush(stdout);
                sleep(10);
                printf("Child3: pid = %d, woke up, exiting\n", getpid());
                return 0;
            } else {
                int status;
                waitpid(pid3, &status, 0);
                printf("Child2: pid = %d, child3 (pid %d) exited\n", getpid(), pid3);
                sleep(10);
                printf("Child2: pid = %d, woke up, exiting\n", getpid());
                return 0;
            }
        } else {
            int status;
            waitpid(pid2, &status, 0);
            printf("Child1: pid = %d, child2 (pid %d) exited\n", getpid(), pid2);
            sleep(10);
            printf("Child1: pid = %d, woke up, exiting\n", getpid());
            return 0;
        }
    } else {
        int status;
        waitpid(pid1, &status, 0);
        printf("Parent: pid = %d, child1 (pid %d) exited\n", getpid(), pid1);
        return 0;
    }
}
```

```
// ---- original parent ----
int status;
waitpid(pid1, &status, 0);
printf("Original parent: pid = %d, child1 (pid %d) exited\n", getpid(), pid1);
}

return 0;
}
```

```
$ gcc -o rec_fork rec_fork.c
$ ./rec_fork
```



Total ~30s wall clock: each parent's own sleep(10) only starts after its child already exited

6. fork + exec — child replaces itself with a different program

exec_prog.c

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("exec_prog: pid = %d, starting count\n", getpid());
    fflush(stdout);

    for (int i = 1; i <= 10; i++) {
        printf("%d\n", i);
        fflush(stdout);
        sleep(1);
    }
}
```

```
    return 0;
}
```

fork_exec.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("fork_exec: parent pid = %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("fork_exec: child pid = %d, about to exec\n", getpid());
        fflush(stdout);

        execl("./exec_prog", "exec_prog", NULL);

        perror("execl failed");
        return 1;
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("fork_exec: child (pid %d) exited\n", pid);
    }

    return 0;
}
```

```
$ gcc -o exec_prog exec_prog.c
$ gcc -o fork_exec fork_exec.c
$ ./fork_exec
```

While it's running (~10 seconds), check the relationship in another terminal:

```
$ ps -o pid,ppid,cmd -C exec_prog
$ pstree -p $(pgrep fork_exec)
$ cat /proc/$(pgrep exec_prog)/status | grep -E 'Pid|PPid'
```



Same pid throughout the middle and right box — `exec()` replaces the memory image in place, it does not create a new process

The pid printed by `fork_exec: child pid = ...` is the exact same pid that `exec_prog: pid = ...` prints. `exec()` replaces the calling process's text/data/heap/stack with the new binary's, keeping the same pid, ppid, open file descriptors, and signal dispositions. `ps` and `pstree` will still show a normal parent/child *process* relationship (`fork_exec`—`exec_prog`), even though only one `fork()` and one `exec()` occurred — not two forks.